GLOBAL TECH INTERVIEW

MASTER LIBRARY

JAVA EDITION

Complete Computer Science • Algorithms • Software Engineering

A structured reference and preparation system for interviews from Graduate / Fresher through Senior and Staff Engineer.

Includes: concepts, algorithm families, decision patterns, Java implementation templates, complexity analysis, system design, core CS, engineering practices and a curated problem bank.

Version 1.0 • 13 November 2026

How to use this library

This is a broad master syllabus and working reference, not a promise that every company asks every topic. Interview expectations vary by role, level, team and region.

Three layers: (1) Core: expected in many software engineering interviews; (2) Advanced: useful for stronger product-company, infrastructure and specialist roles; (3) Specialist: role-dependent topics that should be prioritized only when relevant.

Study loop: understand the concept → derive the approach → implement in Java without copying → analyze complexity → test edge cases → explain trade-offs aloud → revisit after 1 day, 1 week and 1 month.

Java track: use Java 17+ syntax unless a target company specifies another version. Know the standard library and be able to write clean code without relying on obscure APIs.

A problem is complete only when you can

• Explain the problem and constraints in your own words.

• Identify a brute-force solution and improve it logically.

• State time and auxiliary-space complexity, including assumptions.

• Implement independently and test empty, singleton, duplicate, sorted, reverse-sorted and boundary cases.

• Explain why the chosen data structure or algorithm is correct.

• Discuss alternatives, trade-offs and possible follow-up variations.

Complexity notation

Notation	Meaning	Typical interpretation
O(f(n))	Asymptotic upper bound	Worst-case upper-bound discussion commonly uses Big-O.
Ω(f(n))	Asymptotic lower bound	A lower bound on growth.
Θ(f(n))	Tight asymptotic bound	Both upper and lower bounds match.
Amortized	Average cost over a sequence	Dynamic-array append is amortized O(1).
Expected	Expectation over randomness	Expected randomized quicksort time O(n log n).

Master roadmap and priority tiers

Tier	What to master	Suggested outcome
Tier 1 — Core	Arrays, strings, hashing, two pointers, sliding window, binary search, sorting, recursion, linked lists, stacks, queues, trees, heaps, graphs, greedy, core DP, Big-O, Java collections.	Solve unseen easy/medium problems and explain complexity.
Tier 2 — Strong	Tries, DSU, monotonic structures, topological sort, shortest paths, MST, interval DP, bitmask DP, segment/Fenwick trees, advanced string matching, concurrency, SQL, OS, networks.	Handle harder variants and role-specific follow-ups.
Tier 3 — Specialist	Flow, matching, HLD, centroid decomposition, persistent structures, suffix automaton, FFT/NTT, computational geometry, distributed consensus, JVM internals, GPU/parallel systems.	Prepare only when target role or interview signal justifies it.

Suggested sequence: foundations → core DSA patterns → trees/graphs/DP → core CS → design and engineering → advanced/specialist topics → company- and role-specific practice.

1. DSA foundations and Java toolkit

Problem-solving workflow

• Clarify input size, value ranges, duplicates, ordering, mutation rules and expected output.

• Choose a model: sequence, set/map, tree, graph, interval, state-space or optimization.

• Start with brute force; identify repeated work, expensive lookup, redundant state or ordering.

• Select a pattern and prove its invariant or recurrence.

• Estimate complexity before coding; check integer overflow and recursion depth.

• Test counterexamples and explain the solution as if teaching another engineer.

Java collections and common costs

Type	Core operations / typical cost	Use cases and cautions
int[] / T[]	Index O(1); search O(n); insert/delete middle O(n)	Fixed-size contiguous storage; excellent locality.
ArrayList	get O(1); append amortized O(1); middle insert/remove O(n)	Resizable array; avoid repeated front insertion.
LinkedList	Known-node insert/remove O(1); indexed access O(n)	Rarely best for algorithm problems; pointer overhead.
HashMap / HashSet	Expected get/put/contains O(1); worst-case can degrade	Frequency, lookup, deduplication; implement equals/hashCode correctly.
TreeMap / TreeSet	get/put/remove O(log n)	Sorted keys, predecessor/successor, range views.
PriorityQueue	peek O(1); offer/poll O(log n)	Heap; Java default is min-heap.
ArrayDeque	add/remove at both ends amortized O(1)	Preferred stack/queue implementation over legacy Stack.
StringBuilder	append amortized O(1); insert may be O(n)	Efficient repeated string construction.

Java implementation habits

• Use int for bounded values and long for potentially large sums/products.

• Use Integer.compare(a, b), Long.compare(a, b) in comparators; avoid subtraction that can overflow.

• Use StringBuilder for repeated concatenation in loops.

• Use ArrayDeque for stack/queue; use PriorityQueue with Comparator for heaps.

• Know equals/hashCode contract for custom keys.

• Understand pass-by-value (including object references), immutability, generics and exception behavior.

• Avoid accidental $O(n^2)$ string concatenation, repeated sorting, and recursion on very deep inputs.

2. Algorithm and problem-solving pattern catalog

Pattern family	Methods to know	Representative problems
Arrays / scanning	Single pass, running min/max, prefix/suffix, difference array, cyclic placement, in-place marking	Maximum subarray; product except self; first missing positive; range updates.
Hashing	Frequency map, index map, set membership, prefix-sum + map, grouping by canonical key	Two Sum; Subarray Sum Equals K; Longest Consecutive Sequence; Group Anagrams.
Two pointers	Opposite ends, same direction, fast/slow, partition pointers	Two Sum II; 3Sum; Container With Most Water; Remove Duplicates.
Sliding window	Fixed window, variable window, frequency-count window, monotonic deque	Longest substring without repeats; Minimum Window Substring; Sliding Window Maximum.

Pattern family	Methods to know	Representative problems
Binary search	Exact search, lower/upper bound, rotated arrays, binary search on monotonic answer	First/last position; Search Rotated Array; Koko Eating Bananas; Median of Two Sorted Arrays.
Sorting / selection	Merge sort, quicksort, heapsort, counting/radix sort, quickselect	Inversions; Kth largest; meeting rooms; custom ordering.
Intervals / sweep line	Sort by start/end, merge, event ordering, active-set sweep	Merge Intervals; Meeting Rooms; skyline-style problems.
Monotonic stack / queue	Next/previous greater-smaller, contribution counting, sliding extrema	Daily Temperatures; Largest Rectangle; Sum of Subarray Minimums.
Recursion / backtracking	Choose-explore-unchoose, pruning, duplicate skipping, constraint propagation	Subsets; permutations; Combination Sum; N-Queens; Sudoku.
Divide and conquer	Split, solve subproblems, combine; recursion tree	Merge sort; quicksort; inversion count; closest pair.
Greedy	Sort by a criterion, locally optimal choice, exchange argument, invariant	Activity Selection; Jump Game; Gas Station; interval scheduling.
Dynamic programming	State, transition, base case, order; memoization/tabulation/space optimization	Knapsack; LIS; LCS; edit distance; grid DP; interval DP.
Bit manipulation	Masks, XOR, bit counts, subset enumeration, bitmask state	Single Number; subsets by mask; maximum XOR; bitmask DP.
Tree traversal	DFS preorder/inorder/postorder, BFS level order, iterative traversal	Diameter; LCA; views; serialize/deserialize; path sums.
Graph traversal	BFS, DFS, multi-source BFS, connected components, cycle detection	Islands; clone graph; bipartite; shortest path in unweighted graph.
Graph ordering / connectivity	Topological sort, DSU, SCC, bridges, articulation points	Course Schedule; Accounts Merge; Critical Connections.
Shortest paths	BFS, 0-1 BFS, Dijkstra, Bellman-Ford, Floyd-Warshall, DAG shortest path	Network Delay Time; Cheapest Flights; all-pairs shortest paths.
Spanning trees / flow	Prim, Kruskal, max-flow/min-cut, bipartite matching	Min Cost to Connect Points; max-flow network; assignment.
Heaps / K-way merge	Top-K, two heaps, merge sorted streams, event scheduling	Top K Frequent; Median Stream; Merge K Lists.
Trie / string algorithms	Prefix trie, KMP, Z, rolling hash, Aho-Corasick, suffix structures	Implement Trie; Word Search II; pattern matching; repeated substring.
Range queries	Fenwick tree, segment tree, lazy propagation, sparse table, persistent tree	Range sum/min/max with updates; inversion counting.
Search in state space	BFS, DFS, bidirectional BFS, A*, IDA*, branch and bound	Word Ladder; shortest path puzzles; route/state search.
Randomized / streaming	Random pivot, reservoir sampling, randomized hashing, streaming sketches	Kth selection; sample from stream; approximate distinct counting.
Math / number theory	GCD, sieve, modular arithmetic, fast power, combinatorics, primality	Modular exponentiation; prime queries; CRT; counting problems.

3. Data structures — complete checklist

Structure	Concepts / operations	Typical complexity
Array / dynamic array	Contiguous indexed storage; resizing; prefix sums; difference arrays	O(1) index; insert/delete middle O(n)
Singly / doubly linked list	Pointer manipulation, sentinel nodes, reverse, merge, cycle detection	Access O(n); known-node update O(1)
Stack / queue / deque	LIFO, FIFO, double-ended operations; monotonic variants	Push/pop/queue ops O(1) amortized
Hash table	Hashing, collision handling, resizing, load factor	Expected lookup/update O(1)
Binary heap	Complete tree, heapify, push/pop, indexed heap concept	Peek O(1); push/pop O(log n); build O(n)
BST	Search, insert, delete, predecessor/successor	O(h); balanced O(log n), skewed O(n)
Balanced trees	AVL, Red-Black, Treap, Splay; rotations and invariants	O(log n) for AVL/RB; amortized for Splay
B-Tree / B+ Tree	Multiway balanced search trees used in storage/indexing	Operations O(log n) height; page-oriented
Trie	Prefix tree, terminal marker, prefix counts, compressed trie	O(L) operations for key length L
DSU	Find/union, path compression, union by size/rank	Nearly O(1) amortized: inverse Ackermann

Structure	Concepts / operations	Typical complexity
Segment tree	Build, point/range update, range query, lazy propagation	O(log n) update/query; O(n) space
Fenwick tree	Prefix sums and point updates; coordinate compression	O(log n) update/query; O(n) space
Sparse table	Static idempotent range min/max/gcd queries	O(n log n) build; O(1) query
Order-statistic / persistent trees	Rank/select, historical versions, kth in range	Typically O(log n) operations
Graph representations	Adjacency list/matrix, edge list, weighted graph	List O(V+E) storage; matrix $O(V^2)$
Bloom filter / sketches	Probabilistic membership and streaming summaries	Space-efficient; false positives possible

4. Algorithm families and key algorithms

Family	Algorithms / methods	Complexity / notes
Sorting	Insertion, selection, bubble, merge, quick, heap, counting, radix, bucket, TimSort concept	Comparison sorts generally O(n log n); counting/radix depend on key range/digits.
Searching	Linear, binary, lower/upper bound, exponential, ternary (unimodal), interpolation (assumptions)	Binary O(log n) on sorted/random-access data; linear O(n).
Selection	Quickselect, heap selection, median of medians	Expected O(n) quickselect; deterministic median-of-medians O(n).
Graph traversal	BFS, DFS, iterative DFS, multi-source BFS, topological sort	O(V+E) with adjacency lists.
Shortest path	BFS, 0-1 BFS, Dijkstra, Bellman-Ford, Floyd-Warshall, DAG relaxation	BFS O(V+E); Dijkstra commonly O((V+E)logV); Bellman O(VE); Floyd $O(V^3)$.
MST	Kruskal + DSU; Prim + heap	Kruskal O(E log E); Prim O(E log V) with heap/list.
SCC / low-link	Kosaraju, Tarjan SCC, bridges, articulation points	O(V+E).
Flow / matching	Ford-Fulkerson, Edmonds-Karp, Dinic, Hopcroft-Karp, Hungarian method	Complexity depends on algorithm and graph; learn assumptions and capacity types.
String matching	Naive, KMP, Z, Rabin-Karp, Boyer-Moore, Aho-Corasick	KMP/Z O(n+m); rolling hash expected fast but collision-aware.
Suffix structures	Suffix array, LCP, suffix automaton, suffix tree concept	Often O(n log n) construction for common suffix-array methods; variants differ.
DP optimization	Bitset DP, monotone queue optimization, divide-and-conquer DP, Knuth, CHT	Only valid under specific transition/monotonicity conditions.
Computational geometry	Orientation, line intersection, sweep line, convex hull, closest pair	Convex hull O(n log n); exact robustness matters.
Number theory	Euclid, extended Euclid, sieve, segmented sieve, Miller-Rabin, CRT	Sieve O(n log log n); modular methods depend on integer sizes.
Matrix / algebra	Binary exponentiation, matrix exponentiation, Gaussian elimination	Fast exponentiation O(log e) multiplications.
Randomized	Randomized quicksort/quickselect, reservoir sampling, randomized treap	Expected bounds; account for adversarial inputs and seeds.

5. Java implementation templates

Templates are starting points. Adapt types, validation and overflow handling to the actual constraints.

Binary search — first index with value >= target

```
static int lowerBound(int[] a, int target) {
    int lo = 0, hi = a.length; // half-open [lo, hi)
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] < target) lo = mid + 1;
        else hi = mid;
    }
    return lo; // may equal a.length
}
```

Time O(log n), space O(1). Requires sorted input. The half-open interval avoids special handling for empty arrays.

Prefix sum — range sum

```
static long[] prefix(int[] a) {
    long[] p = new long[a.length + 1];
    for (int i = 0; i < a.length; i++) {
        p[i + 1] = p[i] + a[i];
    }
    return p;
}
// Sum inclusive range [left, right]:
// p[right + 1] - p[left]
```

Build O(n), each range query O(1), extra space O(n). Use long if sums may exceed int.

Sliding window — longest substring without repeated characters

```
static int longestUniqueSubstring(String s) {
    Map<Character, Integer> last = new HashMap<>();
    int left = 0, best = 0;
    for (int right = 0; right < s.length(); right++) {
        char c = s.charAt(right);
        if (last.containsKey(c)) {
            left = Math.max(left, last.get(c) + 1);
        }
        last.put(c, right);
        best = Math.max(best, right - left + 1);
    }
    return best;
}
```

Expected O(n) time and O(min(n, alphabet)) map space. This Java char-based version iterates UTF-16 code units; use code points if full Unicode code-point semantics are required.

Disjoint Set Union

```
static class DSU {
    int[] parent, size;
    DSU(int n) {
        parent = new int[n];
        size = new int[n];
        for (int i = 0; i < n; i++) {
            parent[i] = i;
            size[i] = 1;
        }
    }
    int find(int x) {
        if (parent[x] != x) parent[x] = find(parent[x]);
        return parent[x];
    }
    boolean union(int a, int b) {
        a = find(a); b = find(b);
        if (a == b) return false;
        if (size[a] < size[b]) { int t = a; a = b; b = t; }
        parent[b] = a;
        size[a] += size[b];
        return true;
    }
}
```

Near-constant amortized operations: O(α(n)) with path compression and union by size.

Breadth-first search — unweighted graph

```
static int[] bfs(List<List<Integer>> g, int source) {
    int n = g.size();
    int[] dist = new int[n];
    Arrays.fill(dist, -1);
    ArrayDeque<Integer> q = new ArrayDeque<>();
    dist[source] = 0;
    q.add(source);

    while (!q.isEmpty()) {
        int u = q.remove();
        for (int v : g.get(u)) {
            if (dist[v] == -1) {
                dist[v] = dist[u] + 1;
```

```
                q.add(v);
            }
        }
    }
    return dist;
}
```

O(V+E) time and O(V) auxiliary space. For weighted edges, use an appropriate shortest-path algorithm instead.

Dijkstra — non-negative edge weights

```
static long[] dijkstra(List<List<int[]>> g, int source) {
    int n = g.size();
    long INF = Long.MAX_VALUE / 4;
    long[] dist = new long[n];
    Arrays.fill(dist, INF);
    dist[source] = 0;

    PriorityQueue<long[]> pq =
        new PriorityQueue<>(Comparator.comparingLong(x -> x[1]));
    pq.offer(new long[]{source, 0});

    while (!pq.isEmpty()) {
        long[] cur = pq.poll();
        int u = (int) cur[0];
        long d = cur[1];
        if (d != dist[u]) continue;

        for (int[] e : g.get(u)) {
            int v = e[0];
            long w = e[1];
            if (d <= INF - w && d + w < dist[v]) {
                dist[v] = d + w;
                pq.offer(new long[]{v, dist[v]});
            }
        }
    }
    return dist;
}
```

Assume g[u] contains {neighbor, non-negative weight}. Typical binary-heap complexity O((V+E) log V), space O(V+E).

Dynamic programming — 0/1 knapsack

```
static int knapsack01(int[] weight, int[] value, int capacity) {
    int[] dp = new int[capacity + 1];
    for (int i = 0; i < weight.length; i++) {
        for (int c = capacity; c >= weight[i]; c--) {
            dp[c] = Math.max(dp[c],
                             value[i] + dp[c - weight[i]]);
        }
    }
    return dp[capacity];
}
```

O(n × capacity) time, O(capacity) space. Iterate capacity downward so each item is used at most once. This assumes non-negative weights and a manageable capacity.

Topological sort — Kahn's algorithm

```
static List<Integer> topoSort(List<List<Integer>> g) {
    int n = g.size();
    int[] indegree = new int[n];
    for (int u = 0; u < n; u++)
        for (int v : g.get(u)) indegree[v]++;

    ArrayDeque<Integer> q = new ArrayDeque<>();
    for (int i = 0; i < n; i++)
        if (indegree[i] == 0) q.add(i);

    List<Integer> order = new ArrayList<>();
    while (!q.isEmpty()) {
        int u = q.remove();
        order.add(u);
        for (int v : g.get(u))
            if (--indegree[v] == 0) q.add(v);
    }
    if (order.size() != n) return List.of(); // cycle exists
```

```
        return order;
    }
```

O(V+E) time and O(V) extra space. An empty result is used here to signal a cycle; choose a distinct result type if an empty DAG is possible.

6. Choosing the right algorithm

Problem clues	First techniques to consider	Important condition
Sorted data / monotonic predicate	Binary search, two pointers	Prove monotonicity or sorted invariant.
Contiguous subarray / substring	Sliding window, prefix sum, deque	Sliding window often needs non-negative or special constraints.
Repeated overlapping subproblems	Memoization / tabulation DP	Define state and transition precisely.
Unweighted shortest path	BFS	Each edge has equal cost.
Non-negative weighted shortest path	Dijkstra	Negative edges invalidate standard Dijkstra.
Negative edge weights	Bellman-Ford or suitable DAG method	Detect reachable negative cycles where relevant.
All-pairs shortest path, small V	Floyd-Warshall	$O(V^3)$, $O(V^2)$ memory.
Connectivity under edge additions	DSU	Basic DSU does not support arbitrary deletions.
Dependencies / prerequisites	Topological sort	Graph must be a DAG for a full ordering.
Repeated next-greater/smaller queries	Monotonic stack	Maintain and state the stack invariant.
Top K / streaming median	Heap(s)	Choose min/max heap sizes based on required order.
Range query with updates	Fenwick / segment tree	Choose based on operation and algebraic properties.
Prefix dictionary / many shared prefixes	Trie	Memory can be high; consider compressed variants.
Enumerate valid configurations	Backtracking + pruning	Worst case often exponential.
Optimization with choices over time/items	Greedy or DP	Greedy needs a correctness proof.

7. Curated problem bank — core to advanced

Use the problem names below as a minimum structured bank. The same problem may teach several patterns. Links are stable practice destinations where possible; platform pages and availability can change.

Arrays, hashing, two pointers, sliding window

Two Sum • Best Time to Buy and Sell Stock • Contains Duplicate • Product of Array Except Self • Maximum Subarray • Maximum Product Subarray • 3Sum • 4Sum • Container With Most Water • Trapping Rain Water • Subarray Sum Equals K • Longest Consecutive Sequence • Set Matrix Zeroes • Spiral Matrix • Rotate Image • First Missing Positive • Merge Intervals • Insert Interval • Non-overlapping Intervals • Longest Substring Without Repeating Characters • Minimum Window Substring • Permutation in String • Sliding Window Maximum • Largest Rectangle in Histogram • Sum of Subarray Minimums

Binary search, sorting, selection

Binary Search • Search Insert Position • Find First and Last Position • Search in Rotated Sorted Array • Find Minimum in Rotated Sorted Array • Search a 2D Matrix • Koko Eating Bananas • Capacity To Ship Packages Within D Days • Split Array Largest Sum • Median of Two Sorted Arrays • Kth Largest Element in an Array • Count of Smaller Numbers After Self • Count Inversions • Reverse Pairs

Linked lists, stacks, queues

Reverse Linked List • Linked List Cycle • Linked List Cycle II • Merge Two Sorted Lists • Reorder List • Remove Nth Node From End • Intersection of Two Linked Lists • Copy List with Random Pointer • Reverse Nodes in k-Group • LRU Cache • LFU Cache • Valid Parentheses • Min Stack • Evaluate Reverse Polish Notation • Daily Temperatures • Next Greater Element • Car Fleet • Basic Calculator • Decode String

Trees, BST, heaps, tries

Maximum Depth of Binary Tree • Diameter of Binary Tree • Balanced Binary Tree • Binary Tree Level Order Traversal • Binary Tree Right Side View • Construct Binary Tree from Preorder and Inorder • Validate Binary Search Tree • Kth Smallest Element in a BST • Lowest Common Ancestor • Serialize and Deserialize Binary Tree • Binary Tree Maximum Path Sum • Recover Binary Search Tree • Merge k Sorted Lists • Top K Frequent Elements • Find Median from Data Stream • Implement Trie • Word Search II

Graphs

Number of Islands • Clone Graph • Max Area of Island • Pacific Atlantic Water Flow • Course Schedule • Course Schedule II • Graph Valid Tree • Number of Connected Components • Redundant Connection • Accounts Merge • Is Graph Bipartite? • Word Ladder • Rotting Oranges • 01 Matrix • Network Delay Time • Cheapest Flights Within K Stops • Path With Minimum Effort • Min Cost to Connect All Points • Critical Connections in a Network • Alien Dictionary

Dynamic programming

Climbing Stairs • House Robber • House Robber II • Decode Ways • Coin Change • Coin Change II • Longest Increasing Subsequence • Longest Common Subsequence • Edit Distance • Word Break • Partition Equal Subset Sum • Target Sum • Unique Paths • Minimum Path Sum • Longest Palindromic Subsequence • Palindromic Substrings • Distinct Subsequences • Interleaving String • Burst Balloons • Best Time to Buy and Sell Stock with Cooldown • Best Time to Buy and Sell Stock IV • Regular Expression Matching • Wildcard Matching • Maximum Students Taking Exam

Greedy, intervals, math, bit manipulation

Jump Game • Jump Game II • Gas Station • Task Scheduler • Hand of Straights • Partition Labels • Minimum Number of Arrows to Burst Balloons • Meeting Rooms II • Candy • Single Number • Counting Bits • Reverse Bits • Sum of Two Integers • Maximum XOR of Two Numbers in an Array • Pow(x, n) • Sqrt(x) • GCD / LCM exercises • Modular exponentiation • Prime sieve queries • Reservoir Sampling

Advanced / specialist practice

Maximum Flow / Minimum Cut implementation • Bipartite Matching • Strongly Connected Components • Bridges and Articulation Points • Eulerian Path / Circuit • A* shortest path • Heavy-Light Decomposition • Centroid Decomposition • Persistent Segment Tree • Suffix Array + LCP • Aho-Corasick multi-pattern search • Convex Hull • Closest Pair of Points • FFT / NTT polynomial multiplication • Knuth Optimization • Divide-and-Conquer DP Optimization

8. Core Computer Science interview syllabus

Operating Systems

• Processes vs threads; process states; context switching; user mode vs kernel mode.

• CPU scheduling: FCFS, SJF/SRTF, Priority, Round Robin; throughput, waiting, turnaround and response time.

• Concurrency: race conditions, critical sections, mutex, semaphore, monitor, condition variable, atomic operations.

• Deadlocks: four necessary conditions, prevention, avoidance, detection and recovery; Banker’s algorithm.

• Memory: virtual address spaces, paging, segmentation, page tables, TLB, page faults, demand paging, copy-on-write.

• Page replacement: FIFO, LRU, Optimal, Clock; Belady’s anomaly; working set and thrashing.

• File systems, I/O, buffering, caching, interrupts, system calls and basic Linux process tools.

DBMS and SQL

• Relational model, keys, constraints, joins, aggregation, subqueries, CTEs and window functions.

• Normalization: 1NF, 2NF, 3NF, BCNF; functional dependencies and denormalization trade-offs.

• Transactions and ACID; isolation levels; dirty/non-repeatable/phantom reads; locks and MVCC.

• Indexes: B+ tree, hash index, composite/covering indexes, selectivity, query plans and index trade-offs.

• Concurrency control, deadlocks, write-ahead logging, recovery, replication and partitioning.

• SQL practice: joins, top-N per group, running totals, gaps/islands, deduplication and ranking.

Computer Networks

• OSI and TCP/IP models; Ethernet, IP addressing, subnetting, ARP, ICMP and routing.

• TCP handshake/teardown, sequence numbers, retransmission, congestion control, flow control and UDP.

• DNS resolution, HTTP methods/status codes/headers, cookies, sessions, caching and TLS.

• HTTP/1.1 vs HTTP/2 vs HTTP/3; REST basics; WebSockets; proxies and load balancers.

• Latency, bandwidth, throughput, packet loss, MTU, NAT and common debugging tools.

OOP and language fundamentals

• Encapsulation, abstraction, inheritance and polymorphism; composition vs inheritance.

• Overloading vs overriding; interfaces vs abstract classes; dynamic dispatch.

• SOLID principles, cohesion, coupling, dependency inversion and object lifecycle.

• Generics, collections, equality, immutability, exceptions, serialization and reflection basics.

9. Java and JVM engineering

• JDK vs JRE vs JVM; source-to-bytecode compilation; class loading and class-loader delegation.

• Primitive types, wrappers, autoboxing, pass-by-value, String pool, String immutability and StringBuilder.

• Collections Framework internals: ArrayList resizing, HashMap hashing/collision handling/tree bins concept, HashSet, TreeMap, PriorityQueue and ArrayDeque.

• equals() / hashCode() contract; Comparable vs Comparator; immutable keys.

• Generics, bounded types, wildcards, type erasure and common pitfalls.

• Exceptions: checked vs unchecked, try-with-resources, custom exceptions.

• Streams and lambdas: map/filter/reduce, collectors, laziness and parallel-stream cautions.

• Concurrency: Thread, Runnable, ExecutorService, Future, CompletableFuture, synchronized, volatile, locks, atomics, concurrent collections.

• Java Memory Model: happens-before, visibility, ordering, safe publication and data races.

• JVM memory areas, stack vs heap, garbage collection concepts, allocation, leaks and profiling.

• Modern Java features: records, sealed classes, pattern matching and switch expressions according to target JDK.

Concurrency concepts to explain

Concept	Meaning / interview angle
Race condition	Outcome depends on timing/interleaving of concurrent operations.
Atomicity	Operation appears indivisible; compound operations are not automatically atomic.
Visibility	One thread's write becomes observable to another under memory-model rules.
Deadlock	Threads wait cyclically for resources; discuss lock ordering and timeouts.
Livelock / starvation	Threads remain active without progress / a thread is indefinitely denied resources.
Thread pool	Bounded worker reuse; discuss queue size, rejection policy and shutdown.
ConcurrentHashMap	Concurrent access with implementation-specific coordination; do not assume every compound sequence is atomic.

10. Software engineering, APIs, testing and delivery

Design and code quality

• SOLID: Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation, Dependency Inversion.

• Design patterns: Strategy, Factory, Builder, Adapter, Decorator, Observer, Command, State, Template Method, Proxy, Singleton (with trade-offs).

• Clean code: meaningful names, small cohesive methods, explicit invariants, error handling and separation of concerns.

• Refactoring, code review, technical debt, backwards compatibility and API versioning.

Testing

• Unit, integration, contract, end-to-end, regression, smoke, performance, load, stress and chaos testing.

• Test doubles: fake, stub, mock and spy; test behavior rather than private implementation details.

• Boundary-value analysis, equivalence classes, property-based testing and fuzzing.

• Deterministic tests, time control, flaky-test diagnosis and test-data management.

Backend and web fundamentals

• HTTP request lifecycle, REST resource modeling, status codes, validation, pagination and idempotency.

• Authentication vs authorization; sessions, tokens, OAuth 2.0 / OpenID Connect concepts, password hashing and least privilege.

• Input validation, SQL injection, XSS, CSRF, SSRF, secrets management and secure defaults.

• Caching: client, CDN, reverse proxy, application and database cache; TTL, invalidation and stampede.

• Messaging: queues vs streams, delivery semantics, retries, dead-letter queues, ordering and idempotent consumers.

DevOps / production readiness

• Git branching, merge/rebase, conflict resolution, pull requests and release tagging.

• Build tools: Maven / Gradle; dependency management; reproducible builds.

• CI/CD pipeline, artifact repositories, containers, Docker basics and Kubernetes concepts.

• Observability: structured logs, metrics, traces, dashboards, alerts, SLI/SLO/SLA and incident response.

• Reliability: timeouts, retries with backoff/jitter, circuit breakers, bulkheads, rate limits and graceful degradation.

11. System design — LLD, HLD and distributed systems

Low-Level Design (LLD)

• Clarify requirements and use cases; identify entities, responsibilities and relationships.

• Define class/interface boundaries, invariants, state transitions and public APIs.

• Apply SOLID and patterns only when they reduce coupling or improve changeability.

• Discuss persistence, concurrency, error handling, extensibility and testability.

• Practice: parking lot, elevator, library, vending machine, meeting scheduler, splitwise-style ledger, rate limiter, cache and notification service.

High-Level Design (HLD)

• Requirements: functional, non-functional, scale, availability, latency, consistency, durability and security.

• Back-of-the-envelope estimation: QPS, storage, bandwidth, cache size, read/write ratio and growth.

• Architecture: clients, API gateway, services, databases, caches, queues, object storage, search and CDN.

• Data design: relational vs NoSQL, schema, indexes, partition key, sharding, replication and data lifecycle.

• Reliability: redundancy, failover, replication lag, retries, idempotency, disaster recovery and multi-region design.

• Trade-offs: consistency vs availability under partitions, synchronous vs asynchronous work, push vs pull, SQL vs NoSQL.

Distributed systems concepts

Topic	Concepts to master
Consistency	Strong, eventual, read-your-writes, monotonic reads, linearizability and causal consistency.
Consensus	Paxos/Raft concepts, leader election, quorum, terms/epochs and failure assumptions.
Replication	Leader-follower, multi-leader, synchronous/asynchronous replication and lag.
Partitioning	Hash/range partitioning, consistent hashing, hotspots, rebalancing and skew.
Transactions	2PC, sagas, outbox pattern, idempotency keys and compensating actions.
Messaging	At-most-once, at-least-once, effectively-once processing; ordering and deduplication.
Resilience	Timeouts, retries, circuit breakers, bulkheads, backpressure and load shedding.
Coordination	Distributed locks, leases, fencing tokens, clocks and failure detection.

System design problem bank

• URL shortener; pastebin; news feed; chat/messaging; notification service; file storage and sync.

• Video streaming; photo sharing; search autocomplete; web crawler; search engine.

• Ride-hailing dispatch; food delivery; ticket booking; hotel booking; payment system.

• Rate limiter; distributed cache; API gateway; metrics/logging platform; feature flag service.

• Job scheduler; workflow engine; distributed task queue; collaborative editor.

• Design a database/indexing service; object store; CDN; recommendation service.

12. Role-specific extensions

Role family	Additional preparation
Backend / Platform	Java internals, APIs, SQL, caching, queues, distributed systems, observability, reliability.
Frontend	JavaScript/TypeScript, browser runtime, DOM, rendering, accessibility, state management, web performance and frontend system design.
Full Stack	Frontend + backend + database + API security + deployment + end-to-end debugging.
Data Engineering	SQL, data modeling, batch/stream processing, Kafka concepts, Spark, orchestration, warehouses, data quality and distributed storage.
ML / AI Engineering	Python ecosystem, statistics, ML fundamentals, evaluation, feature/data pipelines, serving, monitoring, GPU basics and model systems.
Infrastructure / SRE	Linux, networking, OS, concurrency, distributed systems, capacity planning, incident response and reliability.
Database / Storage	Index structures, query execution, transactions, WAL, MVCC, replication, compaction and storage engines.
Security Engineering	Threat modeling, cryptography foundations, authentication, authorization, secure coding, network/app security and incident analysis.
Compiler / Runtime	Lexing, parsing, AST, type systems, intermediate representation, optimization, code generation, garbage collection and JIT.
GPU / Systems	C/C++ fundamentals, memory hierarchy, parallelism, synchronization, SIMD, GPU execution model and performance profiling.

13. Behavioral and communication interviews

• Prepare concise STAR stories: Situation, Task, Action, Result; quantify impact honestly.

• Stories to prepare: conflict, failure, ambiguity, ownership, difficult debugging, prioritization, mentoring, learning and customer impact.

• Explain decisions, alternatives rejected, risks, measurable outcome and what you would improve.

• For senior levels: technical strategy, cross-team influence, mentoring, incident leadership, architecture evolution and trade-offs.

• Never invent impact or claim ownership of work you did not do.

14. Level-by-level preparation expectations

Level	Typical interview emphasis	Preparation evidence
Graduate / Fresher	DSA fundamentals, coding correctness, complexity, OOP, basic OS/DBMS/CN, projects and communication.	Solve core easy/medium patterns independently; explain fundamentals.
SDE / SWE II	Broader medium/hard DSA, practical design, debugging, testing, APIs, databases and ownership.	Handle follow-ups, trade-offs, clean implementation and production reasoning.
Senior Engineer	System design depth, distributed systems, reliability, architecture, mentoring, execution and cross-team work.	Lead design discussions, identify failure modes, justify trade-offs and deliver outcomes.
Staff / Principal	Org-level technical direction, multi-system architecture, strategy, influence, risk management and long-term evolution.	Show durable technical judgment, leverage across teams and strategic impact.

These are broad patterns, not universal leveling rules. A company may use different titles, loops and expectations.

15. Company-specific research workflow

• Read the official role description and identify the actual team, level and required technologies.

• Check the company's official candidate/interview guidance first.

• Use public problem lists and community reports as signals, not guarantees; reports can be stale or biased.

• Map each interview round to a skill: coding, design, domain knowledge, behavioral or practical exercise.

• Build a role-specific gap list and prioritize by likelihood, importance and current weakness.

• After each mock interview, record the question type, missed insight, complexity issue, communication gap and next revision date.

Useful resources: LeetCode (leetcode.com), NeetCode (neetcode.io), CP-Algorithms (cp-algorithms.com), GeeksforGeeks (geeksforgeeks.org), Tech Interview Handbook (techinterviewhandbook.org), System Design Primer (github.com/donnemartin/system-design-primer), official company careers/interview pages, Java documentation (docs.oracle.com/en/java/).

16. Weekly revision and practice operating system

Day	Primary focus	Practice output
1	Arrays, hashing, prefix sums, two pointers	2 new problems + 1 revision
2	Sliding window, strings, binary search	2 new problems + explain invariants
3	Linked lists, stacks, queues, heaps	1 implementation + 2 problems
4	Trees, BST, tries	1 traversal/design exercise + 2 problems

Day	Primary focus	Practice output
5	Graphs, DSU, shortest paths, MST	1 graph implementation + 2 problems
6	DP, greedy, intervals, bit manipulation	2 problems; derive recurrence/proof
7	Timed mixed set, CS fundamentals, review	Mock interview + error-log review

For a 3–4 hour study block: 45–60 min concepts, 90–120 min problem solving, 30–45 min implementation/review, and 15–30 min verbal explanation. Adjust for college/work and recovery.

Error log template

Date / problem	Missed concept	Why first attempt failed	Correct invariant / approach	Revisit
				1 day / 1 week / 1 month
				1 day / 1 week / 1 month
				1 day / 1 week / 1 month

17. Final readiness checklist

Coding and DSA

- ■ Can recognize core patterns without hints.
- ■ Can derive brute force and optimized approaches.
- ■ Can implement common structures and algorithms in Java.
- ■ Can state time/space complexity and justify correctness.
- ■ Can test boundary cases and discuss alternatives.

Core CS

- ■ Can explain process/thread, scheduling, memory and deadlocks.
- ■ Can write non-trivial SQL and explain indexes/transactions.
- ■ Can explain TCP, HTTP, DNS, TLS and common network failures.
- ■ Can explain OOP, SOLID and Java collection/JVM basics.

Engineering

- ■ Can design testable APIs and modular code.
- ■ Can debug a production-style issue methodically.
- ■ Understands CI/CD, logs, metrics, traces and safe deployment.
- ■ Understands security fundamentals and reliability patterns.

System design

- ■ Can clarify requirements and estimate scale.
- ■ Can choose data stores and partitioning with reasons.
- ■ Can explain consistency, replication, caching and queues.
- ■ Can identify bottlenecks, failure modes and mitigations.

Interview communication

- ■ Can think aloud without rambling.

• ■ Can ask useful clarifying questions.

• ■ Can explain trade-offs and accept feedback.

• ■ Has honest STAR stories and project deep dives.

Appendix A — Advanced algorithm inventory

• Dynamic programming: digit DP, tree DP, rerooting DP, bitmask DP, profile DP, interval DP, probability DP, LIS optimization, divide-and-conquer optimization, Knuth optimization, convex hull trick, monotone queue optimization.

• Graphs: bridges, articulation points, biconnected components, SCC, Euler tour/path, Eulerian circuit, Hamiltonian path concepts, max flow, min cut, bipartite matching, min-cost max-flow, 2-SAT, centroid decomposition, heavy-light decomposition, virtual tree.

• Strings: prefix function, KMP, Z-function, rolling hash, Rabin-Karp, Aho-Corasick, suffix array, LCP, suffix automaton, suffix tree, Manacher's algorithm, palindrome tree (Eertree).

• Data structures: implicit treap, persistent segment tree, lazy segment tree, merge-sort tree, wavelet tree, link-cut tree, rope, interval tree, order-statistic tree, Bloom filter, Count-Min Sketch, HyperLogLog.

• Math: Euclidean and extended Euclidean algorithms, modular inverse, Chinese Remainder Theorem, Euler/Fermat theorems, sieve variants, Miller-Rabin, Pollard Rho, combinatorics, fast doubling Fibonacci, matrix exponentiation, Gaussian elimination, FFT/NTT.

• Geometry: orientation/cross product, line-segment intersection, convex hull (monotonic chain/Graham scan), rotating calipers, sweep line, closest pair, point-in-polygon.

• Search/optimization: A*, IDA*, bidirectional search, branch and bound, minimax, alpha-beta pruning, simulated annealing, local search, approximation algorithms.

Appendix B — Interview problem-solving answer format

```
1. Restate the problem and constraints.
2. Ask clarifying questions.
3. Give a straightforward baseline.
4. Identify bottleneck and propose an improved approach.
5. Explain data structures and invariant / recurrence.
6. State time and space complexity.
7. Implement clean Java code.
8. Dry-run a normal case and edge cases.
9. Discuss limitations, alternatives and follow-ups.
```

Appendix C — Important accuracy notes

• No checklist can guarantee every question asked by every company; interviews vary by team, level and year.

• A topic being advanced does not make it automatically high priority. Master core patterns before specialist algorithms.

• Complexities in this guide are typical bounds under stated data-structure and input assumptions; implementation details can change them.

• For Java, confirm the target JDK and permitted libraries for the actual interview environment.

• The uploaded DSA checklist is a useful starting inventory; this library expands it with broader CS and software engineering coverage rather than claiming the original sheet alone is complete.